

Lab 4

TASK 1: File Encryption with OpenSSL

1. To complete the given task, I've created a file named `message.txt`
2. After, I've generated RSA Private Key

[illegible]

3. Using private key, I've extracted the Public Key using following command

```
PS C:\Users\JONI> openssl rsa -pubout -in private.pem -out public.pem
writing RSA key
PS C:\Users\JONI>
```

4. Then encrypted message.txt with RSA public key and used RSA private key to decrypt it

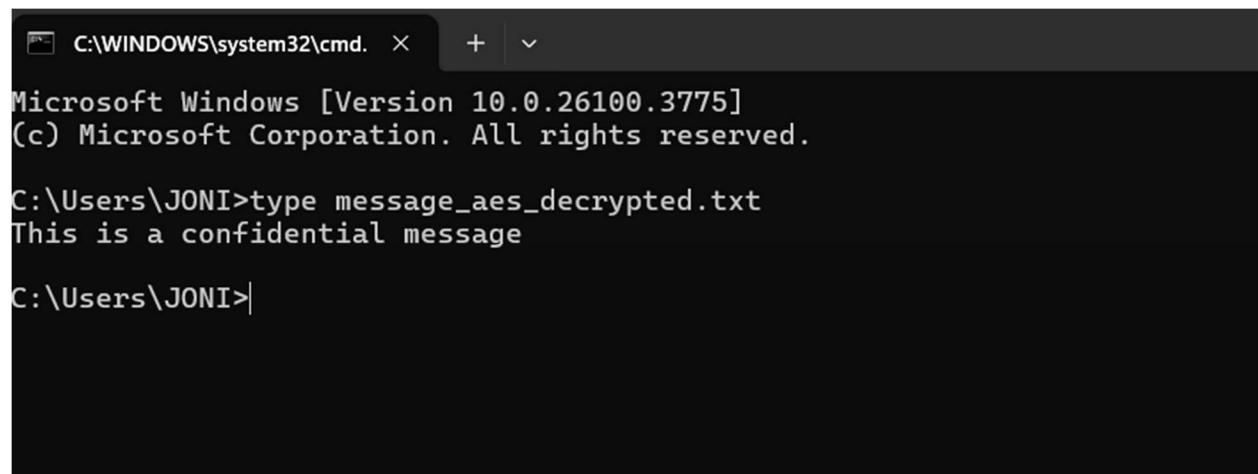
```
PS C:\Users\JONI> openssl rsautl -encrypt -inkey public.pem -pubin -in message.txt -out message_rsa_encrypted.bin
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
PS C:\Users\JONI> openssl rsautl -decrypt -inkey private.pem -in message_rsa_encrypted.bin -out message_rsa_decrypted.tx
t
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
PS C:\Users\JONI> openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out message_rsa_encrypted.bin
PS C:\Users\JONI> openssl pkeyutl -decrypt -inkey private.pem -in message_rsa_encrypted.bin -out message_rsa_decrypted.t
xt
```

- Generated a 256-bit AES key to encrypt and decrypt with AES-256-CBC:

```
PS C:\Users\JONI> openssl rand 32 > aes_key.bin
PS C:\Users\JONI> openssl rand 16 > aes_iv.bin
```

```
iv undefined
PS C:\Users\JONI> certutil -encodehex aes_key.bin aes_key.hex 0
Input Length = 70
Output Length = 156
CertUtil: -encodehex command completed successfully.
PS C:\Users\JONI> certutil -encodehex aes_iv.bin aes_iv.hex 0
Input Length = 38
Output Length = 110
CertUtil: -encodehex command completed successfully.
PS C:\Users\JONI> |
```

```
PS C:\Users\JONI> openssl enc -aes-256-cbc -in message.txt -out message_aes_encrypted.bin -K 4B35F9F61A2D4E8E43F9B5C1A11D17A489E3579D0C5D7318F5ED2E8821D3847A -iv 5A1F9C4D3E2B1A0F4739A6DCB281675A
PS C:\Users\JONI> openssl enc -d -aes-256-cbc -in message_aes_encrypted.bin -out message_aes_decrypted.txt -K 4B35F9F61A2D4E8E43F9B5C1A11D17A489E3579D0C5D7318F5ED2E8821D3847A -iv 5A1F9C4D3E2B1A0F4739A6DCB281675A
PS C:\Users\JONI>
```



```
C:\WINDOWS\system32\cmd.  ×  +  ∨  
Microsoft Windows [Version 10.0.26100.3775]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\JONI>type message_aes_decrypted.txt  
This is a confidential message  
  
C:\Users\JONI>
```

Summary:

RSA is best for encrypting small data and secure key exchanges, but it's slow. AES is fast and used for bulk encryption with a shared key. AES is symmetric; RSA is asymmetric. AES is used after RSA key exchange in TLS.

TASK 2: SSL/TLS in HTTPS

Part 1: Inspect Website Certificate (OpenSSL)

1. In windows terminal, where OpenSSL is installed, I run the command - openssl s_client -connect google.com:443 – which connects my computer to the server google.com over HTTPS (port 443) and simulates the initial SSL/TLS handshake.

```

PS C:\Users\JONI> openssl s_client -connect google.com:443
Connecting to 172.217.17.110
CONNECTED(00000200)
depth=2 C=US, O=Google Trust Services LLC, CN=GTS Root R4
verify error:num=20:unable to get local issuer certificate
verify return:1
depth=1 C=US, O=Google Trust Services, CN=WE2
verify return:1
depth=0 CN=*.google.com
verify return:1
---
Certificate chain
 0 s:CN=*.google.com
  i:C=US, O=Google Trust Services, CN=WE2
  a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA256
  v:NotBefore: Mar 20 11:18:50 2025 GMT; NotAfter: Jun 12 11:18:49 2025 GMT
 1 s:C=US, O=Google Trust Services, CN=WE2
  i:C=US, O=Google Trust Services LLC, CN=GTS Root R4
  a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA384
  v:NotBefore: Dec 13 09:00:00 2023 GMT; NotAfter: Feb 20 14:00:00 2029 GMT
 2 s:C=US, O=Google Trust Services LLC, CN=GTS Root R4
  i:C=BE, O=GlobalSign nv-sa, OU=Root CA, CN=GlobalSign Root CA
  a:PKEY: EC, (secp384r1); sigalg: sha256WithRSAEncryption
  v:NotBefore: Nov 15 03:43:21 2023 GMT; NotAfter: Jan 28 00:00:42 2028 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIINyDCCDQAgAwIBAgIRAJpZ42kgVIHMCS+ecU3PC0IwCgYIKoZIzj0EAwIwOzEL
MAKGA1UEBhMCVVMxHjAcBgNVBAoTFUdvdv2dsZSBUcnVzdCBTZXJ2aWNLczEMMAoG
A1UEAxMDV0UyMB4XDTE1MDMyMDEyMTg1MFoXDTE1MDYxMjExMTg0VowFzEVMBMG
A1UEAwMKi5nb29nbGUuY29tMFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEgd6I
SGMACxtgt1960Z0hqFDQwFDr+D2QX5I4x3xFlfJz0HFsMc+S7wAyXulkbfr5qMD
c88lWZhfdtc8Bl6eJqOCDA0wggwJMA4GA1UdDwEB/wQEAwIHgDATBgNVHSDDDAK
BggrBgEFBQcDATAMBgNVHRMBAf8EAjAAMB0GA1UdDgQWBBS5i+6w8Lf5j+UrlhI
bLuIg02vkTafBgNVHSMGDAWgBR1vsR3ron2RDd9z7FoHx0a69w0WTBYBggrBgEF
BQcBAQRMEowIQYIKwYBBQUHMAAGFWH0dHA6Ly9vLnBraS5nb29nL3dLMjAlBggr
BgEFBQcwAoYzAHR0cDovL2kucGtPLmdvb2cvd2UyLmNydDCCCEQGA1UdEQSCCdsW
ggNggwQLmdvb2dsZS5jb22CFiouYXBwZW5naW5lLmdvb2dsZS5jb22CCSouYmRu
LmRldoIVKi5vcmlnaW4tdGVzdC5iZG4uZGV2ghIqLmNsb3VhLmdvb2dsZS5jb22C
GCouY3Jvd2Rzb3VyY2UuZ29vZ2xLLmNvbYIYKi5kYXRhY29tcHV0ZS5nb29nbGUu
Y29tggsqLmdvb2dsZS5jYyILKi5nb29nbGUuY2yCDiouZ29vZ2xLLmNvLmLugg4q
Lmdvb2dsZS5jby5qcIIOKi5nb29nbGUuY28udWuCDyouZ29vZ2xLLmNvbS5hcoIP
Ki5nb29nbGUuY29tLmFigg8qLmdvb2dsZS5jb20uYnKCDyouZ29vZ2xLLmNvbS5j
b4IPKi5nb29nbGUuY29tLm14gg8qLmdvb2dsZS5jb20dHKKCDyouZ29vZ2xLLmNv
bS52boILKi5nb29nbGUuZGWCyouZ29vZ2xLLmVzggqLmdvb2dsZS5mcoILKi5n
b29nbGUuHwCCyouZ29vZ2xLLmL0ggqLmdvb2dsZS5ubIILKi5nb29nbGUuGyC
CyouZ29vZ2xLLnB0gg8qLmdvb2dsZWFWaXMuY26CESouZ29vZ2xldmLkZW8uY29t
ggwqLmdzdGF0aWMuY26CECouZ3N0YXRpYy1jb15jb22CD2dvb2dsZWnuYXBwcy5j
boIRKi5nb29nbGVjbWwFwCHMuY26CEWdvb2dsZWFWcHMtY24uY29tghMqLmdvb2ds
ZWFWcHMtY24uY29tggsna2VjbWwFwCHMuY26CDiouZ2tLY25hcHBzLmNughJnb29n
bGVkb3dubG9hZHMuY26FCouZ29vZ2xLZG93bmxxvYWRzLmNughByZWNhchRjaGEu
bmV0LmNughIqLmJlY2FwdGNoYS5uZXQuY26CEHJlY2FwdGNoYS1jb15uZXSCeiou
cmVjYXB0Y2hhLWnuLm5ldIILd2lkZXZpbmUuY26CDSoud2lkZXZpbmUuY26CEWft
cHByb2pLY3Qub3JnLmNughMqLmFtcHByb2pLY3Qub3JnLmNughFhbXBwcm9qZWNo
Lm5ldC5jboITKi5hbXBwcm9qZWNoLm5ldC5jboIXZ29vZ2xLLWFuYXx5dGJjcy1j
bi5jb22CGSouZ29vZ2xLLWFuYXx5dGJjcy1jb15jb22CF2dvb2dsZWFWkc2Vydmlj
ZXMtY24uY29tghkqLmdvb2dsZWFWkc2VydmljZXMtY24uY29tghFnb29nbGV2YWRz
LWnuLmNvbYITKi5nb29nbGV2YWRzLWnuLmNvbYIRZ29vZ2xLYXBpcy1jb15jb22C
EyouZ29vZ2xLYXBpcy1jb15jb22CFWdvb2dsZW9wdGltaxpLLWnuLmNvbYIXKi5n
b29nbGVvchRpbWl6ZS1jb15jb22CEmRvdWJsZWNSaWNrLWnuLm5ldIUKi5kb3Vi
bGVjbGJjay1jb15uZXSCGcouZmxzLmRvdWJsZWNSaWNrLWnuLm5ldIUKi5nLmRv

```

-----END CERTIFICATE-----

■■■■■

■■■■■

■■■■■ ■■■■■ ■■■■■

■■■■■ ■■■■■ ■■■■■

```
20
---
Post-Handshake New Session Ticket arrived:
SSL-Session:
  Protocol : TLSv1.3
  Cipher   : TLS_AES_256_GCM_SHA384
  Session-ID: BC9382630DF637DA6643D8B63E0ED1CFEC3A0C708A492080DCACD5EBB618BC9E
  Session-ID-ctx:
  Resumption PSK: B8E04BDCCB316881C620EAC51B1B20BA7E9AB918A9A551EF5BA0E0427AE0EAF17EB9D21B21E5BFB503026B38D3E3E8C3
  PSK identity: None
  PSK identity hint: None
  SRP username: None
  TLS session ticket lifetime hint: 172800 (seconds)
  TLS session ticket:
0000 - 02 cf 03 88 65 96 2f e6-0a 87 13 0e 57 56 e0 2e   ....e./....WV..
0010 - 09 19 13 76 55 60 cc bc-f9 3f 48 7b 06 94 68 ca   ...vU`...?H{..h.
0020 - 72 6a 98 06 98 2b f9 f2-7a 3b df 79 5b e2 08 08   rj...+.z;;y[...
0030 - b5 25 22 c5 0c be 5b c5-55 10 84 df b2 c2 32 60   .%"...[.U.....2`
0040 - bb d5 cc 25 4d 3d fc a6-c4 04 ec 2d 4b 62 c1 de   ...%M=.....-Kb..
0050 - f5 e7 bc ea cb 3d 94 d8-fa 6c 3c 3c 14 23 7f c3   ....=...l<<.#..
0060 - bc 8b 09 b8 e0 ac 18 64-c3 9a 3d a9 8b 42 49 ae   ....d...=.BI.
0070 - e5 58 9f fd c2 c2 ad 36-96 02 79 76 68 1f c6 a0   .X....6..yvh...
0080 - 86 93 6e 00 ba 27 0d 36-1e 31 db 42 95 14 67 67   ..n..'.6.1.B..gg
0090 - cb c1 aa 9e 69 34 93 e1-21 9d fd 86 3c d3 c8 47   ....i4...!...<..G
00a0 - 4a fc da 55 ac bc 59 d2-48 41 6d 7e 74 a3 f5 46   J..U..Y.HAm~t..F
00b0 - 35 e9 45 4a dd 69 9a 23-90 1f 9a 74 eb da 8e 00   5.EJ.i.#...t....
00c0 - 99 11 3c 89 56 ec db 60-6e 6b fa fc c2 37 9e d7   ..<.V..`nk...7..
00d0 - 37 cb 61 e4 11 c7 e6 63-9b 4a e7 9f 08 b6 98 4a   7.a....c.J.....J
00e0 - c5 4f f9 e2 92 16 1d 69-86 70 78 53 4f 46 91 b6   .O.....i.pxSOF..
00f0 - 4a cd 44 ba a2 c0                               J.D...

  Start Time: 1744618337
  Timeout : 7200 (sec)
  Verify return code: 20 (unable to get local issuer certificate)
  Extended master secret: no
  Max Early Data: 14336
---
read R BLOCK
---
Post-Handshake New Session Ticket arrived:
SSL-Session:
  Protocol : TLSv1.3
  Cipher   : TLS_AES_256_GCM_SHA384
  Session-ID: 0A4F24704EFF3C2D298EC539DA74D0726F00BF93CF00E2F998F09B97F5A0C996
  Session-ID-ctx:
  Resumption PSK: 61B578996594B09FC75A34F04B85F60517CC2BF517DEF944CEFC42A025B7DECE1C28AE1F290E8DA1BD08481D3D228FF6
  PSK identity: None
  PSK identity hint: None
```

```

read R BLOCK
---
Post-Handshake New Session Ticket arrived:
SSL-Session:
    Protocol : TLSv1.3
    Cipher : TLS_AES_256_GCM_SHA384
    Session-ID: 0A0F24704EFF3C2D298EC539DA74D0726F00BF93CF08E2F998F09B97F5A0C996
    Session-ID-ctx:
    Resumption PSK: 61B578996594B09FC75A34F04B85F60517CC2BF517DEF944CEFC42A025B70CE1C28AE1F290E80A1B080481D30228FF6
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    TLS session ticket lifetime hint: 172800 (seconds)
    TLS session ticket:
0000 - 02 cf 03 88 65 96 2f e6-0a 87 13 0e 57 56 e0 2e ...../.....WV..
0010 - 49 2b f7 07 05 15 fa 11-a8 1a 0c f9 53 7a 36 00 I+.....Sz6.
0020 - 3e 01 8a d3 0d f7 cb e3-74 0b 1b 7c c1 74 86 5d >.....t..|t.]
0030 - 1a a7 7f d4 d0 d5 f9 85-2f dc 18 d9 e4 04 33 29 ...../.....S)
0040 - 31 0e 86 1c ca 23 6c 84-3f d9 f9 5b 79 92 25 0b 1...#L.?.[y.%
0050 - 7e bf 19 3f bc b3 a2 20-56 9b e8 f5 54 e7 02 b8 "...?... V...T...
0060 - b6 73 27 81 48 15 ed b1-f3 c1 20 b3 7a 69 22 52 .s'.H.....z1'R
0070 - d3 0b 4a 08 32 37 9c 1e-ad 82 c2 8a 3b 48 89 c4 ...J.27.....H.-
0080 - e3 83 d9 96 8c c3 c5 e6-0b 1b 2d 91 29 6a 9d 46 .....-...}.F
0090 - 65 7d ac 89 38 40 ec c3-81 99 b7 4d a7 57 e0 c8 e}..00.....M.W..
00a0 - 76 4d 0b d4 b4 38 42 58-75 2a 72 24 b7 cd 6b 3b vM...8BXu*$.&.k;
00b0 - a6 9f 33 38 41 7c 3e-d5 28 52 70 e8 26 18 ad ...3BA]>..Rx.&..
00c0 - 84 d8 c0 f5 ee d7 4a 40-ef 88 52 a0 ee 22 e4 7e .....WK..R...-
00d0 - 7f 9b c9 c4 7c 69 35 f7-37 5d 50 9f 2e 18 84 14 ....[i5.7].....
00e0 - 66 08 3b b6 e1 dc 04 15-a1 51 a4 53 4f 46 b5 3f f;.....Q.SOF.?
00f0 - d2 70 c6 c2 82 17 .p....

Start Time: 1744618337
Timeout : 7200 (sec)
Verify return code: 20 (unable to get local issuer certificate)
Extended master secret: no
Max Early Data: 14536
---
read R BLOCK
HTTP/1.0 400 Bad Request
Content-Type: text/html; charset=UTF-8
Referer-Policy: no-referrer
Content-Length: 1555
Date: Mon, 14 Apr 2025 08:12:18 GMT

<!DOCTYPE html>
<html lang=en>
  <meta charset=utf-8>
  <meta name=viewport content="initial-scale=1, minimum-scale=1, width=device-width">
  <title>Error 400 (Bad Request)!!1</title>
  <style>
    *{margin:0;padding:0}html,code{font:15px/22px arial,sans-serif}html{background:#fff;color:#222;padding:15px}body{margin:7% auto 0;max-width:390px;min-height:180px;padding:30px 0 15px}*
  > body{background:url(//www.google.com/images/errors/robot.png) 100% 5px no-repeat;padding-right:205px}p{margin:11px 0 22px;overflow:hidden}ins{color:#777;text-decoration:none}a img{border:
  </style>

  <!DOCTYPE html>
  <html lang=en>
    <meta charset=utf-8>
    <meta name=viewport content="initial-scale=1, minimum-scale=1, width=device-width">
    <title>Error 400 (Bad Request)!!1</title>
    <style>
      *{margin:0;padding:0}html,code{font:15px/22px arial,sans-serif}html{background:#fff;color:#222;padding:15px}body{margin:7% auto 0;max-width:390px;min-height:180px;padding:30px 0 15px}*
    > body{background:url(//www.google.com/images/errors/robot.png) 100% 5px no-repeat;padding-right:205px}p{margin:11px 0 22px;overflow:hidden}ins{color:#777;text-decoration:none}a img{border:
    r:0}@media screen and (max-width:777px){body{background:none;margin-top:0;max-width:none;padding-right:0}}#logo{background:url(//www.google.com/images/branding/googlelogo/2x/googlelogo_color_150x54dp.png)
    or_150x54dp.png) no-repeat;margin-left:-5px}@media only screen and (min-resolution:192dpi){#logo{background:url(//www.google.com/images/branding/googlelogo/2x/googlelogo_color_150x54dp.png)
    ) no-repeat 0% 0%/100% 100%;-moz-border-image:url(//www.google.com/images/branding/googlelogo/2x/googlelogo_color_150x54dp.png) no-repeat;-webkit-background
    image:linear-gradient(to right,transparent 48%,#4285f4 48%,#4285f4 52%,transparent 52%);border:none}img{display:block;margin:auto;max-width:100%;height:150px}}
    </style>
    <a href=//www.google.com/><span id=logo aria-label=Google></span></a>
    <p><b>400.</b> <ins>Thatŕċŕŕs an error.</ins>
    <p>Your client has issued a malformed or illegal request. <ins>Thatŕċŕŕs all we know.</ins>
4C490000:error:0A000126:SSL routines:unexpected eof while reading:ssl\record\rec_layer_s3.c:696:
PS C:\Users\JONI> |

```

This information includes Certificate chain, Subject & Issuer, Cipher suite, TLS version.

1. TLS Handshake Summary:

- The client initiated a connection to google.com on port 443.

- The handshake showed the certificate chain:

Leaf certificate for .google.com (ECDSA with SHA256)

Intermediate certificate (CN=WE2)

Root certificate (CN=GTS Root R4)

- TLS Version: TLSv1.3

- Cipher Suite: TLS_AES_256_GCM_SHA384

- Server public key: 256-bit EC (prime256v1)
- Session was successfully negotiated.
- Certificate verification error (expected): "unable to get local issuer certificate" -this happens because local CA root certificates aren't provided to OpenSSL by default.

2. Certificate Details:

- Subject: CN=*.google.com
- Issuer: CN=WE2, O=Google Trust Services
- Validity: Mar 20 – Jun 12, 2025
- Signature Algorithm: ecdsa-with-SHA256
- Key Exchange Group: X25519 (from TLS 1.3 negotiation)

3. MITM Protection in TLS:

- TLS provides strong protection against MITM via:
 - Certificate validation (authenticates the server)
 - Forward secrecy (with ephemeral Diffie-Hellman like X25519)
 - Encryption of all communication post-handshake

4. Explanation of the Error:

- The `verify error:num=20:unable to get local issuer certificate` simply means OpenSSL couldn't verify the full certificate chain because no trusted root certificates were provided locally.
- This does **not** mean the certificate is invalid -browsers and systems with full trust stores would verify it correctly.

5. Real-World Use:

- TLS is the cryptographic backbone of HTTPS - it ensures:
 - Data integrity
 - Confidentiality
 - Server authenticity

Step 2: Capture TLS Handshake in Wireshark

1. In Wireshark I selected the network interface and filtered traffic using `- tcp.port == 443`. Started the capture and opened webpage google.com

1219	24.470646	192.168.0.5	2.16.221.156	TCP	54	51658 → 443 [RST, ACK] Seq=1177 Ack=6402 Win=0 Len=0
1225	24.818415	192.168.0.5	93.186.137.210	TCP	66	51660 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1232	24.861116	93.186.137.210	192.168.0.5	TCP	66	443 → 51660 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1384 SACK_PERM WS=128
1233	24.861201	192.168.0.5	93.186.137.210	TCP	54	51660 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1234	24.862001	192.168.0.5	93.186.137.210	TLSv1.3	1888	Client Hello (SHA256WithRSAEncryption)
1240	24.904707	93.186.137.210	192.168.0.5	TCP	54	443 → 51660 [ACK] Seq=1 Ack=1385 Win=64128 Len=0
1241	24.904707	93.186.137.210	192.168.0.5	TCP	54	443 → 51660 [ACK] Seq=1 Ack=1835 Win=64128 Len=0
1242	24.905914	93.186.137.210	192.168.0.5	TLSv1.3	1514	Server Hello, Change Cipher Spec, Application Data
1243	24.905914	93.186.137.210	192.168.0.5	TCP	1514	443 → 51660 [PSH, ACK] Seq=1461 Ack=1835 Win=64128 Len=1460 [TCP PDU reassembled in 1261]
1244	24.905914	93.186.137.210	192.168.0.5	TCP	1230	443 → 51660 [PSH, ACK] Seq=2921 Ack=1835 Win=64128 Len=1176 [TCP PDU reassembled in 1261]
1245	24.906802	192.168.0.5	93.186.137.210	TCP	54	51660 → 443 [ACK] Seq=1835 Ack=4097 Win=65280 Len=0

Wireshark capture showing a complete TLS 1.3 handshake between the client and server. The "Client Hello" initiates the secure session, followed by "Server Hello" and encrypted handshake messages, transitioning into secure application data exchange.

<https://github.com/JChoniadze/Lab-4>